The Paper, Piece by Piece

A screenshot of every part of our research paper, each followed by what it means in simple words

Mihir Bhati (24BCE5341) · Ahana Sharan (24BCE5334) · Guide: Vijayaprabakaran K

VIT Chennai

How to read this. Each box is cut straight from the paper. Under it: **In short** says what that part is telling the reader, and **Words used here** explains every technical word in it. A dashed line inside a picture means the text continues in the next column or on the next page. For the longer, section-by-section explanation and the teacher questions, see `Research_Paper_Explained.pdf`.

Five ideas to keep in your head while reading

1. **The task:** turn a sentence into an **intent** (what the user wants) and **entities** (the details), or say "not supported".
2. **The design:** there are 12 ways (designs) to set up the AI for this. Choosing well matters (13–16 points difference).
3. **Our agent:** tries only 4 of the 12 on 100 practice sentences and keeps the best by a fixed rule.
4. **Fair testing:** everything was frozen before the real test; P3 was a whole project kept secret until then.
5. **The result:** best design found every time; wins on P1 and P2; on P3 slightly worse on intents (could be luck) but much better on entities.

Page 1: title, abstract, introduction

Title and authors

Automated Design of Task-Specific Intent and Slot Pipelines with a Small Language-Model Agent: A Controlled Evaluation on MASSIVE

Mihir Bhati*
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
24BCE5341
mihir.bhati2024@vitstudent.ac.in

Ahana Sharan*
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
24BCE5334
ahana.sharan2024@vitstudent.ac.in

Vijayaprabakaran K
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
vijayaprabakaran.k@vit.ac.in

From the paper

In short: What the paper is about in one line: a small AI agent that automatically designs the part of an assistant that understands requests (intents and slots), tested carefully on the MASSIVE dataset. Then who wrote it. The * means Mihir and Ahana contributed equally.

Words used here:

- **Automated design:** the computer makes the design choices, not a person.
- **Task-specific:** built for one particular app (e.g. a smart-home app).
- **Intent and slot:** intent = what the user wants (`alarm_set`); slot = a detail (time = "seven").
- **Pipeline:** the full chain from the user's sentence to the final answer.
- **Small language-model agent:** a program that uses a small AI model (3 billion parameters) to make decisions.
- **Controlled evaluation:** a fair test with fixed rules decided in advance.
- **MASSIVE:** a public dataset of assistant requests from Amazon.

Abstract and index terms

***Abstract*—Building the intent and slot component of a new assistant still requires a developer to decide, by trial and error, how the language model is prompted, which examples it sees and how it rejects unsupported requests. We present an agent that makes them from a project specification: a description, an intent inventory, an entity schema and ten labelled utterances per intent. The agent validates and enriches the specification, ranks the options of each design factor, evaluates four of twelve candidate pipelines on 100 development utterances, revises one factor, selects under schema-validity, entity and latency constraints, and exports a hash-verified package. Every planning and runtime call uses one 3-billion-parameter model running on a laptop. On three projects built from MASSIVE, one held out until the protocol was frozen, the agent chose the development-optimal design every time. Against a generic prompt it raised test intent macro-F1 by 2.2 and 4.9 points and entity F1 by 7.3 and 5.5 points on the two development projects, meeting the preregistered success rule; on the held-out project intent F1 fell by 1.3 points (not significant) while entity F1 rose by 14.6 points. It also scored above a k-shot XLM-R baseline on all three projects.**

***Index Terms*—natural language understanding, intent classification, slot filling, out-of-scope detection, language model agents, in-context learning, automated pipeline design**

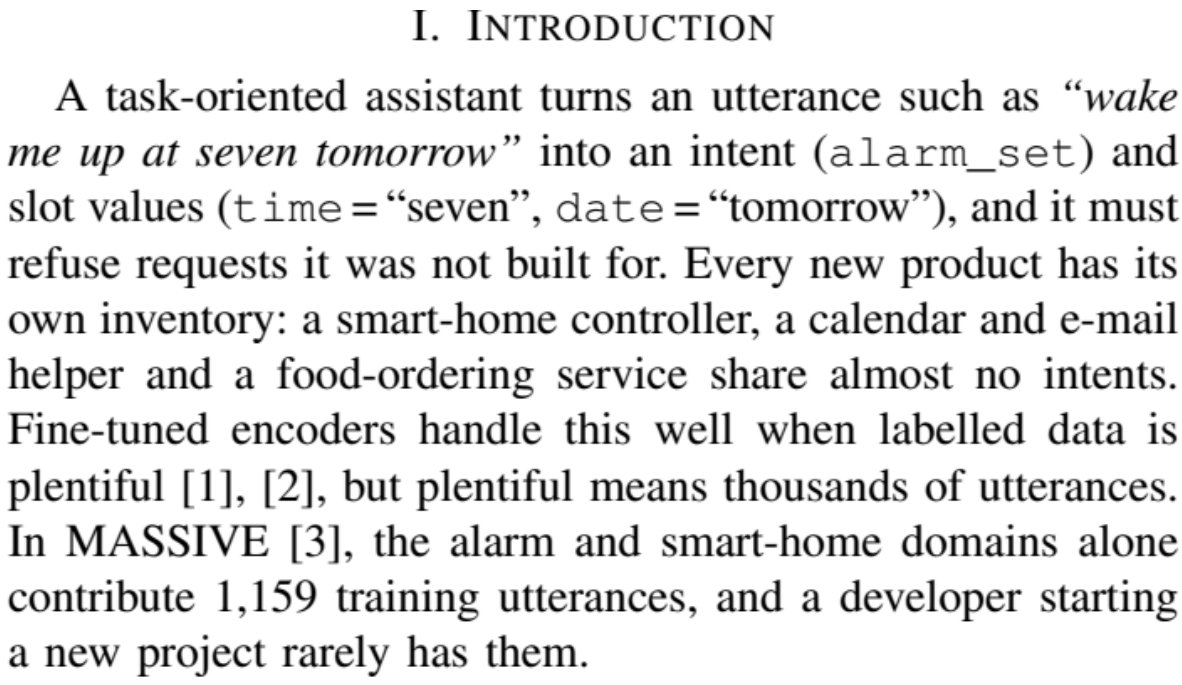
From the paper

In short: The whole paper in 198 words: the problem (choosing prompt, examples and rejection by trial and error), what we built (an agent that does it automatically), how it works (check, rank, test 4 of 12, revise, select, export) and the main numbers. Index terms are search keywords.

Words used here:

- **Project specification:** what the agent is given: description, list of intents, list of entity types, 10 example sentences per intent.
- **Utterance:** one user sentence.
- **Validates and enriches:** checks the input for mistakes, then writes a short description of every label.
- **Design factor:** one of the three choices (how to extract, which examples, how to reject).
- **Development utterances:** practice sentences used to choose (100 per project).
- **Schema validity:** the AI's reply is in the required format.
- **Latency:** how long one request takes.
- **Hash-verified package:** the saved pipeline, with fingerprints that prove nothing was changed.
- **Held out until the protocol was frozen:** P3 stayed locked until all rules and choices were fixed.
- **Development-optimal design:** the best of all 12 designs on the practice set.
- **Macro-F1, points:** the accuracy score (0 to 1); 1 point = 0.01.
- **Preregistered success rule:** the pass/fail rule we fixed before testing.
- **Not significant:** the difference is small enough that it could be luck.
- **k-shot XLM-R:** a trained model (XLM-R) given the same 10 examples per intent.

Introduction, paragraph 1: the problem



I. INTRODUCTION

A task-oriented assistant turns an utterance such as *"wake me up at seven tomorrow"* into an intent (`alarm_set`) and slot values (`time` = "seven", `date` = "tomorrow"), and it must refuse requests it was not built for. Every new product has its own inventory: a smart-home controller, a calendar and e-mail helper and a food-ordering service share almost no intents. Fine-tuned encoders handle this well when labelled data is plentiful [1], [2], but plentiful means thousands of utterances. In MASSIVE [3], the alarm and smart-home domains alone contribute 1,159 training utterances, and a developer starting a new project rarely has them.

From the paper

In short: Explains the task with an example and why it is hard for a new app: every app has different intents, and trained models need thousands of labelled sentences (MASSIVE has 1,159 for alarms and smart home alone).

Words used here:

- **Task-oriented assistant:** an assistant built to do jobs (alarms, emails), not to chat.
- **Refuse requests it was not built for:** say "not supported" for out-of-scope requests.
- **Inventory:** the list of intents one app supports.
- **Fine-tuned encoders:** models such as BERT or XLM-R that learn by training on many labelled examples.
- **[1], [2], [3]:** reference numbers; the full papers are listed at the end.

Introduction, paragraph 2: why design matters

Small instruction-tuned language models (LMs) remove the training step. A 3-billion-parameter model quantised to 4 bits runs on a laptop GPU with 4 GB of memory, answers in

*Equal contribution.

under half a second and can follow a label inventory from a handful of examples. What it does not remove is design work. The developer still has to decide whether intent and slots are requested in one call or two, whether the prompt describes each label, which demonstrations go into the prompt, and whether out-of-scope (OOS) requests are rejected by the model or by a similarity threshold. These choices matter. With the model and data held fixed, the twelve combinations we study span 13.5 points of development intent macro-F1 on one project (0.839 to 0.974) and 16.0 points on another (0.738 to 0.898).

From the paper

In short: Small AI models need no training and run on a laptop, but someone must still decide how to use them. Those decisions change accuracy a lot: the 12 designs range from 0.839 to 0.974 on P1 and 0.738 to 0.898 on P2.

Words used here:

- **Instruction-tuned language model (LM):** an AI trained to follow written instructions.
- **3-billion-parameter, quantised to 4 bits:** a small model, compressed so it fits in 4 GB of laptop memory.
- **Label inventory:** the list of intents.
- **Demonstrations:** solved example sentences placed in the prompt.
- **Out-of-scope (OOS):** a request the app does not support.
- **Similarity threshold:** reject a request if it is not similar enough to any example.
- **Span 13.5 points:** best minus worst design = 0.974 − 0.839 = 0.135.

Introduction, paragraph 3: what others did and what is missing

> Work on automating LM use has concentrated on rewriting instruction text [4]–[6], tuning the demonstrations of a fixed program [7], or replacing the LM with a fine-tuned small model [8]. None of these produces a joint intent and span output with a rejection decision under deployment constraints, and they are rarely compared with random search at the same budget, the standard control for adaptive search [9]. Without that control, the contribution of the LM's reasoning cannot be separated from simply trying several options.

From the paper

In short: Other work rewrites instructions, tunes examples or trains a small model, but none outputs intent + entities + rejection under real limits, and they are rarely compared with random choice. Without that comparison you can't tell if the AI's reasoning helped.

Words used here:

- **Fixed program:** a pipeline whose structure a person already wrote.
- **Joint intent and span output:** giving the intent and the entity positions together.
- **Deployment constraints:** limits a real app has (format must be valid, must be fast).
- **Random search:** trying options chosen at random; the fair baseline for any smart search.
- **Adaptive search:** a search that uses results so far to choose what to try next.

Introduction, paragraph 4 and the three contributions

This paper asks whether an LM agent with a budget of four candidate evaluations can design an intent and slot pipeline that beats a generic prompt, random search, a human designer and a fine-tuned encoder trained on the same examples. The contributions are:

1) A formulation of task-specific NLU pipeline design as constrained selection over a factorised discrete design space, and an agent that analyses a project specification, plans, evaluates, revises, selects and exports a verifiable package (Section III).
2) A factor-wise planning strategy that builds candidates one factor at a time and revises exactly one factor. It reached the best of the twelve designs on development data on all three projects with four evaluations, although the model's own first preferences were wrong on two of the three factors (Section IV-H).
3) A controlled evaluation on three projects derived from MASSIVE, one of them held out until the protocol was frozen, against six baseline families including matched-budget random search, a human designer and fine-tuned XLM-R, with paired bootstrap confidence intervals (Section IV).

From the paper

In short: The research question (can a 4-try agent beat a generic prompt, random search, a human and a trained model?) and what the paper adds: (1) the method and agent, (2) the factor-wise planning idea that found the best design every time, (3) a careful test on three projects.

Words used here:

- **Candidate evaluations:** designs actually tested; the budget is 4.
- **Generic prompt:** the default setup with no design effort (our B1).
- **Constrained selection over a factorised discrete design space:** pick the best from a fixed list of combinations of 3 choices, while obeying rules.
- **Verifiable package:** an output whose files can be checked for changes.
- **Factor-wise planning:** decide each of the 3 choices separately, then test them in a fixed pattern.
- **Baseline families:** the different kinds of systems we compare against.
- **Matched-budget random search:** random search allowed the same 4 tries.
- **Paired bootstrap confidence intervals:** a statistics method to tell real differences from luck.

Pages 2–3: literature survey (15 papers)

Each paragraph is one earlier paper: what it did, then (last sentence) how it connects to our work.

Survey intro and A. MASSIVE [3]

II. Literature Survey

This section focuses on an extensive survey of fifteen works that define the data, models, prompting methods and search procedures our system builds on.

A. NLU datasets and fine-tuned models

FitzGerald et al. [3] released MASSIVE, one million utterances in 51 languages localised by professional translators from the English SLURP corpus [10]. Each utterance carries one of 60 intents across 18 scenarios and span annotations over 55 slot types. Their XLM-R and mT5 baselines are fine-tuned on the full training split. We use the English portion and restrict each project to a few scenarios, which yields small, realistic single-product inventories.

From the paper

In short: MASSIVE is our dataset: 1 million requests in 51 languages, 60 intents, 18 topics, 55 slot types. We use only English and split it into small apps.

Words used here:

- **Survey:** a review of earlier research.
- **Localised by professional translators:** translated and adapted for each language by humans.
- **SLURP corpus [10]:** the English collection MASSIVE was built from.
- **Scenario:** a topic group, like alarm or email.
- **Span annotations:** markings of exactly which words are each slot.
- **mT5:** another model used as a baseline by MASSIVE's authors.
- **Single-product inventories:** intent lists for one app each.

Larson et al. [11]: out-of-scope requests

Larson et al. [11] built CLINC150, 150 intents over ten domains plus a separate set of queries that match none of them. They showed that classifiers with high in-scope accuracy recall few out-of-scope queries, whether rejection is done by a confidence threshold or by an extra "out-of-scope" class trained on examples. For this reason our design space treats rejection as a separate factor and we report OOS F1 separately.

From the paper

In short: A dataset with off-topic questions showed models that are accurate on normal requests still miss most off-topic ones. That is why rejection is one of our three design choices and why we report OOS F1 separately.

Words used here:

- **CLINC150:** their dataset (150 intents).
- **In-scope accuracy:** accuracy on requests the app does support.
- **Recall few out-of-scope queries:** they catch only a small share of off-topic requests.
- **Confidence threshold:** reject if the model is not confident enough.
- **OOS F1:** our score for how well a system refuses off-topic requests.

Chen et al. [1]: BERT for intent and slots

Chen et al. [1] predicted the intent from BERT's sentence representation and the slot tags from its token representations, training both objectives together. On ATIS and Snips, joint training improved intent accuracy, slot F1 and whole-utterance accuracy over earlier recurrent and attention-based models. Joint encoders remain the reference architecture for intent and slot tasks, and our fine-tuned baseline follows the same two-head design.

From the paper

In short: One BERT model with two outputs (one for the intent, one for each word's slot), trained together, beat older models. Our XLM-R baseline uses the same two-output design.

Words used here:

- **BERT:** a famous encoder model from Google.
- **Sentence representation / token representations:** the model's summary of the whole sentence / of each word.
- **Objectives:** what the model is trained to get right.
- **ATIS and Snips:** two older intent-and-slot datasets.
- **Two-head design:** one model, two outputs.

Conneau et al. [2]: XLM-R

> *Conneau et al.* [2] trained XLM-R, a masked language model, on 2.5 TB of filtered web text in 100 languages and reported strong cross-lingual transfer on classification, question answering and named entity recognition. We fine-tune its base version on the ten utterances per intent the agent sees and on each project's full training split.

From the paper

In short: XLM-R is a big encoder trained on web text in 100 languages. We train its base version as baseline B3, on our 10 examples per intent and on the full training data.

Words used here:

- **Masked language model:** learns by guessing hidden words in sentences.
- **2.5 TB of filtered web text:** an enormous amount of training text.
- **Cross-lingual transfer:** learning in one language helps in others.
- **Named entity recognition:** finding names of people, places etc.
- **Fine-tune:** train an existing model further on your own data.

B. Brown et al. [12]: GPT-3 and in-context learning

> *B. In-context learning and example selection*
>
> *Brown et al.* [12] showed that a sufficiently large autoregressive LM can perform a new task from an instruction and a few demonstrations placed in its prompt, without any weight update, and that accuracy rises with the number of demonstrations and with model size. Once a task is specified through the prompt, its contents and the number of calls become design decisions, which our design space enumerates.

From the paper

In short: A big AI can do a new task just from instructions and a few examples in the prompt, with no training. Once the prompt is the program, what goes in it becomes a design decision, which is what our 12 designs are.

Words used here:

- **In-context learning:** learning a task from examples inside the prompt.
- **Autoregressive LM:** a model that writes text one piece at a time.
- **Weight update:** changing the model's numbers (training). Not needed here.
- **Enumerates:** lists all the options.

Liu et al. [13]: choosing good examples

Liu et al. [13] retrieved, for each test input, the training examples nearest to it in a sentence-encoder space and placed them in the prompt. Nearest-neighbour demonstrations beat randomly sampled ones on sentiment analysis, table-to-text generation and open-domain question answering. Our similarity-based example selection applies the same idea with a lexical TF-IDF index, which costs nothing at request time and needs no second model.

From the paper

In short: Putting the examples most similar to the question into the prompt works better than random examples. Our "similarity examples" option does the same with a cheap word-matching method (TF-IDF).

Words used here:

- **Sentence-encoder space:** sentences turned into numbers so similar ones are close together.
- **Nearest-neighbour demonstrations:** the most similar examples.
- **TF-IDF index:** a word-based way to measure how similar two sentences are; rare shared words count more.

C. Zhou et al. [4]: APE (automatic prompt engineer)

C. Automatic prompt and program optimisation

Zhou et al. [4] proposed Automatic Prompt Engineer (APE): an LM writes candidate instructions from input–output pairs, each candidate is scored on held-out examples, and the best is kept or resampled. The generated instructions matched or exceeded human-written ones on most instruction-induction tasks tested. APE's output is a prompt, not a pipeline, for tasks with a single label or string as output.

From the paper

In short: An AI writes many possible instructions, each is scored, and the best is kept; often as good as human-written ones. But it produces one prompt for simple tasks, not a full pipeline.

Words used here:

- **Input–output pairs:** example questions with their answers.
- **Held-out examples:** examples kept aside for scoring.
- **Resampled:** generate more candidates around the good ones.
- **Instruction-induction tasks:** tasks where the model must work out the instruction from examples.

Pryzant et al. [5]: fixing prompts from their mistakes

Pryzant et al. [5] described the errors of a prompt on a minibatch in natural language, treated that description as a "gradient", edited the prompt against it, and kept the best edits with beam search and a bandit procedure. The method improved initial prompts on several binary classification tasks. Our revision phase borrows the idea of showing the model an error report, but confines its answer to one move in a discrete space, so a wrong suggestion costs one evaluation and cannot produce an unrunnable pipeline.

From the paper

In short: Describe a prompt's errors in words, edit the prompt to fix them, keep the best edits. Our revision step also shows the AI an error report, but allows only one move from a fixed list, so a bad suggestion can't break anything.

Words used here:

- **Minibatch:** a small group of examples.
- **"Gradient":** here, a written description of what is wrong (borrowed from training maths).
- **Beam search, bandit procedure:** ways of keeping the most promising options while searching.
- **Binary classification:** yes/no tasks.
- **Discrete space:** a fixed list of options (our 12 designs).

Yang et al. [6]: OPRO (AI as optimiser)

Yang et al. [6] cast optimisation as prompting (OPRO): the meta-prompt lists earlier solutions with their scores and the LM proposes a new one. They applied it to small regression and travelling-salesman instances and to instruction search on GSM8K and Big-Bench Hard. OPRO needs many proposal rounds; our setting allows one, because every candidate evaluation costs about a hundred model calls on development data.

From the paper

In short: Show the AI earlier attempts with their scores and ask for a better one, over many rounds. We can only afford one round of revision because each test costs about 100 AI calls.

Words used here:

- **Meta-prompt:** a prompt that contains past attempts and scores.
- **Travelling-salesman, GSM8K, Big-Bench Hard:** standard test problems (route planning, maths, hard reasoning).

Khattab et al. [7]: DSPy

Khattab et al. [7] introduced DSPy, in which an LM pipeline is written as declarative modules and a compiler bootstraps demonstrations and instructions to maximise a user-supplied metric. Compiled pipelines let small open models compete with larger prompted ones on multi-hop question answering and math word problems. DSPy optimises the contents of a structure the developer writes; our agent chooses the structure, including the number of calls per request and where rejection happens.

From the paper

In short: Write an AI pipeline as building blocks; a tool automatically picks good examples and instructions. DSPy tunes what is inside a structure you wrote; our agent picks the structure itself.

Words used here:

- **Declarative modules:** blocks that say what to do, not how.
- **Compiler bootstraps demonstrations:** automatically creates good examples.
- **Multi-hop question answering:** questions that need several facts combined.

D. Viswanathan et al. [8]: Prompt2Model

D. From task description to deployable system

Viswanathan et al. [8] built Prompt2Model, which takes a task description with a few examples, retrieves related datasets and a pretrained model, generates further training data with an LM and fine-tunes a small model for deployment. It is the closest work to ours in aim, a deployable artefact from a description, but it requires training and was not evaluated on slot filling or out-of-scope rejection.

From the paper

In short: From a task description, finds data, makes more data and trains a small model to deploy. Closest to our goal (description → working system), but needs training and was not tested on slots or rejection.

Words used here:

- **Deployable artefact:** something ready to use in a real app.
- **Pretrained model:** a model already trained on general data.

E. Madaan et al. [14]: Self-Refine

E. Agents and self-correction

Madaan et al. [14] let one LM generate an output, criticise it and rewrite it, repeating until a stopping condition, with no training. The loop improved outputs on seven tasks including dialogue response and code optimisation. We use a narrow form of self-correction at runtime: when an output breaks the schema, the model receives the specific violation and one chance to repair it.

From the paper

In short: The AI writes an answer, criticises it and rewrites it. We use a small version: if a reply breaks the format, the AI gets one chance to repair it.

Words used here:

- **Self-correction:** the model fixing its own output.
- **Schema:** the required reply format.
- **Violation:** the specific way the reply broke the format.

Yao et al. [15]: ReAct

> *Yao et al.* [15] interleaved reasoning traces with actions such as search queries, so that observations from the environment inform the next reasoning step. ReAct improved question answering and fact verification and reduced hallucination compared with reasoning alone. Our agent follows the same pattern at the level of design: it reasons, acts by running a candidate on development data, observes the scores and error report, and acts once more.

From the paper

In short: The AI alternates thinking and acting and learns from what it observes. Our agent does the same at design level: think, test a design, look at the scores, act once more.

Words used here:

- **Reasoning traces:** the model writing out its thinking.
- **Actions / observations:** doing something (like a search) and seeing the result.
- **Hallucination:** the AI making things up.

F. Bergstra and Bengio [9]: random search

F. Search baselines and automated machine learning

Bergstra and Bengio [9] showed that random search finds good hyper-parameters with far fewer trials than grid search, because only a few dimensions usually matter. Random search has since become the baseline any adaptive optimiser must beat at an equal budget. We therefore compare the agent with five seeds of random search over the same twelve designs, with the same four-evaluation budget and the same selection rule.

From the paper

In short: Random search finds good settings faster than trying everything in a grid. It is the baseline any smart search must beat, so we compare the agent with 5 runs of random search using the same 4 tries and the same rule.

Words used here:

- **Hyper-parameters:** settings you choose before training.
- **Grid search:** trying every combination in a table.
- **Adaptive optimiser:** a search that learns from results as it goes.
- **Seed:** a starting number that makes a random choice repeatable.

He et al. [16]: AutoML survey

He et al. [16] surveyed automated machine learning across data preparation, feature engineering, hyper-parameter optimisation and neural architecture search, and organised it by search space, search strategy and evaluation cost. In their terms our system is a pipeline-level AutoML method with a small hand-defined search space, an LM as the search strategy and development-set evaluation as the expensive step.

From the paper

In short: Automated machine learning searches over data steps, features, settings and network designs. In those terms, ours has a small hand-made search space, an AI as the search method, and practice-set testing as the costly step.

Words used here:

- **AutoML:** automating the choices in building machine-learning systems.
- **Neural architecture search:** automatically designing neural networks.
- **Search space / search strategy / evaluation cost:** what you search over / how / how expensive each try is.

G. Research gap

G. Research gap

Table I compares the eight most relevant approaches. Prompt and program optimisers search instruction text or demonstrations for one output metric; fine-tuning approaches need training data and compute; none of them produces intent, character-level slots and a rejection decision together under schema and latency constraints. More importantly for an agent claim, only one of them compares its search with a random-sampling search of the same kind, and none does so for a complete pipeline, so the contribution of the LM's reasoning to pipeline design has not been isolated. The gap we address is therefore: *whether a small local LM agent, given only a project specification and a fixed evaluation budget, chooses better complete intent and slot pipelines than random choice, a generic prompt and a human, when tested under a frozen protocol.*

From the paper

In short: What is missing in earlier work, in one sentence: nobody tested whether a small local AI agent with a fixed budget picks better complete intent + slot pipelines than random choice, a generic prompt and a human, under frozen rules. That is exactly what we test.

Words used here:

- **Research gap:** the open question a paper answers.
- **Character-level slots:** entities given as exact character positions.
- **Frozen protocol:** all rules fixed before the final test.

Table I: comparison with related work

TABLE I
COMPARISON WITH THE MOST RELEVANT PRIOR WORK

Work	What is searched or learned	Labelled data needed	Slot spans	OOS rejection	Deployable artefact	Random-search control
Chen et al. [1]	encoder weights	full training set	yes	no	model	–
FitzGerald et al. [3]	encoder weights (XLM-R, mT5)	full training set	yes	no	model	–
Liu et al. [13]	demonstrations per input	training pool	no	no	no	random examples
Zhou et al. [4]	instruction text	few examples	no	no	prompt	no
Pryzant et al. [5]	instruction text	small labelled set	no	no	prompt	Monte-Carlo sampling
Yang et al. [6]	instruction text	small labelled set	no	no	prompt	no
Khattab et al. [7]	demonstrations and instructions	small labelled set	no	no	compiled program	no
Viswanathan et al. [8]	data and model weights	few examples	no	no	small model	no
This work	**pipeline design (3 factors)**	**10 per intent + 100 dev**	**yes**	**yes**	**hashed package**	**yes, 5 seeds**

From the paper

In short: A side-by-side summary of 8 earlier works and ours. Only our system outputs slot spans, handles out-of-scope requests, gives a verified package and is compared with random search over the same space.

Words used here:

- **What is searched or learned:** what each method changes (model weights, instruction text, examples, or our pipeline design).
- **Labelled data needed:** how many labelled examples it needs.
- **Slot spans:** does it output entities with positions?
- **OOS rejection:** does it refuse off-topic requests?
- **Deployable artefact:** what you get at the end.
- **Random-search control:** was it compared against random choice?

Pages 3–5: proposed methodology

III-A. Problem formulation

III. Proposed Methodology

A. *Problem formulation*

A project is a tuple $P = (D, I, E, K, C)$. D is a free-text description of the product. I is the intent inventory and E the entity schema, which lists the entity types and, for each intent, the types that intent may carry. K is a pool of labelled utterances with $k = 10$ per intent, each annotated with its intent and entity spans. C holds the deployment constraints: a minimum schema validity $S_{\min}$, a minimum entity F1 relative to a reference and a maximum 95th-percentile latency $L_{\max}$. A labelled development set V contains in-scope utterances and OOS utterances drawn from unrelated domains.

A pipeline maps an utterance u either to a rejection or to a pair $(\hat{\imath}, \hat{S})$, where $\hat{\imath} \in I$ and $\hat{S}$ is a set of typed character spans of u. The task is to choose a pipeline design θ from a design space Θ that performs best on unseen requests, while evaluating at most $B = 4$ designs on V and satisfying C.

From the paper

In short: Describes the task in symbols: a project P is made of a description D, intents I, entity schema E, example pool K and constraints C, plus a practice set V. The goal: pick the design θ from the 12 that will work best on new requests, testing at most 4 and obeying the constraints.

Words used here:

- **Tuple P = (D, I, E, K, C):** a bundle of the five inputs.
- **S_min:** minimum share of valid replies (0.98).
- **L_max:** maximum allowed slow-case time per request.
- **95th-percentile latency (p95):** 95% of requests are faster than this.
- **î (i-hat), Ŝ (S-hat):** the predicted intent and predicted spans; the hat means "predicted".
- **θ (theta) and Θ (Theta):** one design, and the set of all 12.
- **B = 4:** the budget of designs to test.

Figure 1: system architecture

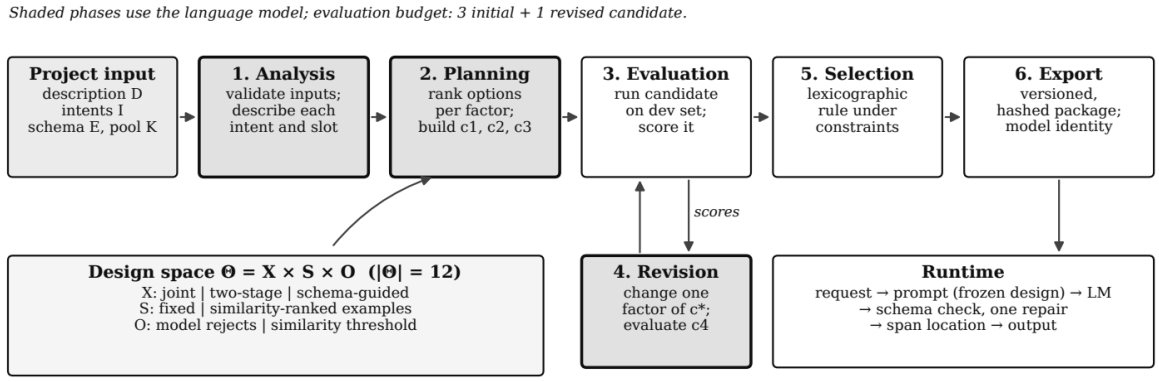


Fig. 1. System architecture. The project specification passes through six phases; the shaded phases call the language model. Evaluation and revision form one loop with a hard budget of three initial candidates and one revised candidate. The exported package is executed by the runtime path on the right.

From the paper

In short: The picture of the whole system. Left to right: project input → 1 Analysis → 2 Planning → 3 Evaluation → 5 Selection → 6 Export. Below: 4 Revision loops back with evaluation (3 tries + 1 revised try). Grey boxes use the AI. The design space box lists the 3 choices; the runtime box shows what happens to one request.

Words used here:

- **Shaded phases:** the steps where the AI model is called.
- **c1, c2, c3, c4:** the four candidate designs tested.
- **Lexicographic rule:** compare by intent score first, then entity score, then speed.
- **Θ = X × S × O (|Θ| = 12):** 3 extraction × 2 example × 2 rejection options = 12 designs.
- **Span location:** turning the entity text into its position in the sentence.

III-B. Design space: extraction and example selection, equation (1)

B. Design space

$\Theta = X \times S \times O$ has three factors and $3 \times 2 \times 2 = 12$ designs. Every design is a fixed, trusted component; the agent chooses among them and never generates executable code.

Extraction strategy X. **Joint**: one call returns the intent and all entities. **Two-stage**: the first call returns only the intent; the second asks only for the entity types that intent carries, with demonstrations of that intent, so each request costs two calls. **Schema-guided**: one call, but the prompt gives a one-line description of every intent and entity type.

Example selection S. **Fixed**: one demonstration per intent, identical for every request. **Similarity**: the $m = 16$ pool utterances nearest to the request by cosine similarity of word unigram and bigram TF-IDF vectors,

$$\mathrm{sim}(u, x) = \frac{\phi(u)^\top \phi(x)}{\|\phi(u)\| \, \|\phi(x)\|}, \tag{1}$$

where ϕ is the TF-IDF map fitted on K with sublinear term frequency.

From the paper

In short: The 12 designs come from 3 choices. Extraction: joint (one call), two-stage (intent first, then its details), or schema-guided (one call with a description of every label). Examples: fixed (same one per intent) or similarity (16 most similar). Equation (1) measures how similar two sentences are.

Words used here:

- **Trusted component:** pre-written, tested code; the agent never writes code.
- **Cosine similarity:** a 0-to-1 score of how much two sentences use the same words (1 = same).
- **ϕ (phi), TF-IDF map:** turns a sentence into a list of word weights.
- **Unigram, bigram:** single words, and pairs of words next to each other.
- **Sublinear term frequency:** a word repeated many times does not count proportionally more.

Rejection policy, equation (2) and the prompt limit

Rejection policy O. **Model label**: the label `none_of_the_above` is offered in the prompt and the model may choose it. **Threshold**: the request is rejected before any model call when $\max_{x \in K} \mathrm{sim}(u, x) < \tau^*$. The threshold is tuned on V only,

$$\tau^* = \arg\max_{\tau \in \mathcal{T}} F_1^{\text{macro}}(I \cup \{\text{OOS}\}; \tau), \qquad (2)$$

where $\mathcal{T}$ is the set of observed similarity values, so the search covers every distinct decision, and ties go to the lower threshold, which rejects fewer in-scope requests.

All designs share a prompt limit of 16 demonstrations and 2,000 input tokens. When the limit is reached, demonstrations are removed first; label definitions are never truncated.

From the paper

In short: Two ways to refuse off-topic requests: let the AI answer `none_of_the_above`, or refuse before calling the AI if the request is not similar enough to any example. Equation (2) picks that cut-off (τ*) on practice data. Prompts may hold at most 16 examples and 2,000 tokens.

Words used here:

- **τ* (tau-star):** the best similarity cut-of
- **arg max:** "the value that gives the highest score".
- **I ∪ {OOS}:** all intents plus the off-topic class.
- **Ties go to the lower threshold:** if two cut-offs score the same, choose the one that refuses fewer real requests.
- **Token:** a piece of a word (about 4 characters).
- **Truncated:** cut short.

III-C/D. Phase 1 (analysis) and start of Phase 2

C. Phase 1: specification analysis

Analysis has a deterministic validation part and an LM enrichment part. Validation checks the specification for duplicate or conflicting names, pool utterances labelled with undeclared intents or entity types, intents with fewer than k examples and entity types that never occur in the pool. A conflict stops the run; the coverage findings are passed on to planning. Enrichment uses two LM calls to write a one-line description for every intent and every entity type. The LM may only describe names that already exist: a description of an unknown name is discarded and a missing one is reported rather than invented, so the supplied schema always takes precedence over model output. The schema-guided strategy places these descriptions in its prompts.

From the paper

In short: Phase 1 checks the input for problems without the AI (duplicate names, unknown labels, too few examples), then uses 2 AI calls to write a one-line description of every intent and entity type. The AI may only describe existing names, never add or rename.

Words used here:

- **Deterministic:** always gives the same result; no AI involved.
- **Coverage findings:** notes such as "this entity type never appears in the examples".
- **Enrichment:** adding the descriptions.
- **Takes precedence:** the given schema always wins over what the AI writes.

Phase 2: the two planners (A1 and A2), equation (3)

D. *Phase 2: candidate planning*

The planner receives the description, the size of the inventory, the coverage findings and a short neutral guide to each factor. We study two planners with the same budget.

Direct planner (A1). The LM is asked to name three designs from the catalogue. Any name not in the catalogue is replaced from a fixed order that spreads the three extraction strategies.

Factor-wise planner (A2). One LM call ranks the options of each factor separately and gives one reason per factor, producing orders $x_1 \succ x_2 \succ x_3$, $s_1 \succ s_2$ and $o_1 \succ o_2$. The three initial candidates are

$$c_1 = (x_1, s_1, o_1), \quad c_2 = (x_1, s_2, o_1), \quad c_3 = (x_2, s_1, o_1). \tag{3}$$

Whatever the model prefers, the alternative example selection and the runner-up extraction strategy are each measured once against its top choice. A malformed ranking for a factor falls back to the declared option order and the candidates are then marked as fallback.

From the paper

In short: A1 just asks the AI to name 3 designs. A2 asks the AI to rank the options of each choice separately, then builds: c1 = all favourites, c2 = favourites but the other example option, c3 = favourites but the second-best extraction. So the alternatives are always tested, even if the AI's favourites are wrong.

Words used here:

- **Planner:** the part that decides which designs to try.
- **x1 ≻ x2 ≻ x3:** x1 is preferred to x2, which is preferred to x3.
- **Runner-up:** second choice.
- **Fallback:** a fixed default used when the AI's answer is unusable.

Phase 3: evaluation

E. Phase 3: evaluation

Each candidate runs on all of V and is scored with the metrics of Section III-I. The phase also writes an error report that lists, for every candidate evaluated so far, its intent macro-F1, entity F1, the fraction of valid replies, the fraction of in-scope requests wrongly rejected and the fraction of predicted entity values not found in the utterance.

From the paper

In short: Each candidate design is run on all 100 practice requests and scored. An error report lists each candidate's scores, share of valid replies, share of real requests wrongly refused, and share of invented entity values.

Words used here:

- **Error report:** the summary shown to the AI before revision.
- **Wrongly rejected:** a supported request the system refused.
- **Not found in the utterance:** the AI returned a detail that is not in the sentence (invented).

Phase 4: revision, equation (4)

F. Phase 4: revision

Let $\hat{c}$ be the best candidate so far under the selection rule below. Its neighbourhood

$$N(\hat{c}) = \{\, c \in \Theta \setminus \mathcal{E} : c \text{ differs from } \hat{c} \text{ in exactly one factor} \,\} \tag{4}$$

excludes the set $\mathcal{E}$ of designs already evaluated. The LM receives the error report and the list of moves in $N(\hat{c})$ and must choose one; an invalid reply falls back to the first move in a fixed factor order. The chosen design is evaluated as the fourth and last candidate. In A1 the revision may name any untried design. The four-evaluation cap is enforced by the agent, not left to the model.

From the paper

In short: The AI sees the error report and picks one change from the best design so far; it may change only one of the three choices and only to an untried design. That design is the 4th and last test. The limit of 4 is enforced by our code, not left to the AI.

Words used here:

- **ĉ (c-hat):** the best candidate so far.
- **N(ĉ), neighbourhood:** untried designs that differ from ĉ in exactly one choice.
- **Θ \ 𝒜:** all designs minus the ones already tested.

Phase 5: selection, equations (5) and (6)

G. Phase 5: selection

Let $\mathcal{C}$ be the evaluated candidates. The eligible set keeps those that meet every declared constraint,

$$\mathcal{C}_{\text{ok}} = \{c \in \mathcal{C} : \text{SV}(c) \geq S_{\min},\ F_1^E(c) \geq f_{\min},\ \hat{L}_{95}(c) \leq L_{\max}\}, \tag{5}$$

and the selected design maximises, in lexicographic order, intent macro-F1, then entity F1, then the negative latency estimate, with the design identifier as a final deterministic tie-break:

$$c^* = \arg\text{lexmax}_{c \in \mathcal{C}_{\text{ok}}} \left(F_1^{\text{macro}}(c),\ F_1^E(c),\ -\hat{L}_{95}(c)\right). \tag{6}$$

No weighted average is used, so a trade-off cannot be hidden in the weights. If $\mathcal{C}_{\text{ok}}$ is empty the agent reports that no candidate met the constraints instead of relaxing them. Cached development runs carry no timing, so $\hat{L}_{95}$ is an estimate from the number of calls per request; the latency of the selected design is measured afterwards without caching. The random-search baseline uses exactly the same rule, so the two differ only in which candidates they evaluate.

From the paper

In short: First remove candidates that break a rule (too many invalid replies, entity score too low, too slow). Then pick the highest intent score; on a tie the higher entity score; then the faster one. No weighted average, and if nothing qualifies the agent says so. Random search uses this same rule.

Words used here:

- $\mathcal{C}$**_ok:** the eligible candidates.
- **SV(c) ≥ S_min:** valid-reply share at least the minimum.
- **arg lexmax:** pick the best by comparing criteria in order, like sorting words alphabetically.
- **Weighted average:** mixing scores with weights; avoided so trade-offs can't be hidden.
- **L̂95:** an estimate of the slow-case time (cached runs have no real timing).

Phase 6: export, and what happens at runtime

H. Phase 6: export and runtime

The package contains the frozen design, the enriched specification, the example pool, τ^*, the prompt templates, the model's name and weights digest, and a SHA-256 hash of every file. Loading recomputes the hashes and refuses to run if any file or the model digest differs, so the deployed pipeline is provably the one that was evaluated.

At runtime a request follows the path at the right of Fig. 1. Under the threshold policy it may be rejected immediately. Otherwise the prompt is built from the frozen design and sent to the LM at temperature zero with structured output. The reply must be an object whose intent is in I or is `none_of_the_above` and whose entities have declared types and string values. A violation triggers one repair call that states the specific error; a second failure is recorded as invalid and kept in every denominator.

From the paper

In short: The final pipeline is saved with a fingerprint of every file and the model's identity; it refuses to run if anything changed. At runtime: optionally refuse by threshold, build the prompt, ask the AI, check the reply's format, allow one repair, and turn entity text into positions.

Words used here:

- **SHA-256 hash:** a file fingerprint; changes if even one character changes.
- **Weights digest:** the fingerprint of the AI model itself.
- **Temperature zero:** the AI always gives its single most likely answer (repeatable).
- **Structured output:** the AI must reply in JSON format.
- **Repair call:** one extra try, telling the AI exactly what was wrong.
- **Kept in every denominator:** a failed reply still counts against the score; nothing is hidden.

The span-location rule

The LM returns entity strings, not offsets, so a deterministic span-location rule converts them. Leading and trailing whitespace and quotation marks are removed; the string is searched in the utterance case-sensitively, then case-insensitively; with several matches the first occurrence not already claimed by another entity is taken. A string that does not occur is *unsupported*: it counts as a false positive and is never fuzzy-matched. Every LM-based system in our evaluation uses this same rule.

From the paper

In short: The AI returns entity text ("seven"), not positions. We find that exact text in the sentence (first exactly, then ignoring capitals). If it is not there, it counts as wrong. No near-matches are accepted, and every AI system uses this same rule.

Words used here:

- **Offsets:** character positions (start and end).
- **Case-sensitive / case-insensitive:** capitals must match / capitals ignored.
- **Unsupported:** an entity text that is not in the sentence (invented).
- **False positive:** a wrong answer the system gave.
- **Fuzzy-matched:** accepting a similar but not exact spelling (we never do this).

III-J. Evaluation metrics, equations (7) and (8)

I. Evaluation metrics

Intent macro-F1 is computed over the in-scope requests and the project's intents:

$$F_1^{\text{macro}} = \frac{1}{|I|} \sum_{i \in I} \frac{2P_i R_i}{P_i + R_i}, \tag{7}$$

where P_i and R_i are the precision and recall of intent i. Rejecting an in-scope request, or returning an invalid output, counts as an error for its true intent. Entity F1 is micro-averaged over exact matches of (type, start, end) triples on in-scope requests: with gold spans G and predicted spans $\hat{S}$,

$$P^E = \frac{|G \cap \hat{S}|}{|\hat{S}|}, \quad R^E = \frac{|G \cap \hat{S}|}{|G|}, \quad F_1^E = \frac{2P^E R^E}{P^E + R^E}. \tag{8}$$

OOS F1 treats rejection as the positive class over all requests. Schema validity SV is the fraction of requests whose final output satisfies the contract after at most one repair. Latency is the median and 95th percentile of wall-clock time per request over uncached runs.

From the paper

In short: How we score. Intent macro-F1: F1 per intent, then averaged. Entity F1: a hit only if type, start and end all match, counted over all requests together. OOS F1: how well refusals are caught. Plus valid-reply share and speed.

Words used here:

- **Precision (P):** of the answers given, how many were right.
- **Recall (R):** of the right answers, how many were found.
- **F1 = 2PR / (P + R):** one number combining both; 1 is perfect.
- **Macro (averaged per intent):** rare intents count as much as common ones.
- **Micro-averaged:** all hits and misses counted together.
- **G ∩ Ŝ:** spans in both the true set and the predicted set (the hits).
- **Wall-clock time:** real seconds measured by a clock.
- **Uncached runs:** the AI really called, not a saved answer replayed.

Pages 5–6: implementation (data, model, baselines, statistics)

IV-A. Dataset and projects

IV. Implementation and Result Analysis

A. Dataset and projects

We use the English (en-US) portion of MASSIVE 1.1 [3], released under CC BY 4.0: 16,521 utterances covering 18 scenarios, 60 intents and 55 slot types. Three projects were defined by grouping related scenarios (Table II). For each project, the pool K takes ten utterances per intent from the official training split, the development set takes 80 in-scope utterances from the official development split, and the test set takes 250 from the official test split. Both add OOS utterances (20 and 50) drawn from eleven scenarios that belong to no project. P3's test set has 167 in-scope and 33 OOS utterances because its official test split holds only 181 in-scope utterances. Sampling used a fixed seed. The entity schema of each project was derived from its training split only, as a developer would declare it before seeing evaluation data; consequently five of P2's 26 entity types never appear in its pool and account for 10.7% of its development gold entities.

From the paper

In short: We use the English part of MASSIVE (16,521 sentences) and build three apps from it. Each gets 10 examples per intent, 100 practice requests and 300 test requests (P3: 200), with off-topic requests from 11 unrelated topics. The entity list comes only from training data, like a real developer.

Words used here:

- **CC BY 4.0:** a free licence: you may use it if you credit the authors.
- **Official training / development / test split:** MASSIVE's own division of its data.
- **Fixed seed:** the random sampling can be repeated exactly.
- **Gold entities:** the correct, human-labelled entities.

Table II: the three projects

TABLE II
THE THREE PROJECTS BUILT FROM MASSIVE EN-US. EVERY DEVELOPMENT SET HAS 80 IN-SCOPE AND 20 OOS UTTERANCES.

Project	Scenarios	$\|I\|$	$\|E\|$	$\|K\|$	Test
P1 home assistant	alarm, iot	12	18	120	250 + 50
P2 productivity	calendar, email, lists	10	26	100	250 + 50
P3 ordering, travel	takeaway, transport	6	19	60	167 + 33

From the paper

In short: Each project's topics, number of intents |I|, number of entity types |E|, number of examples |K|, and test size (in-scope + off-topic).

Words used here:

- **P1 home assistant:** alarms and smart devices; 12 intents.
- **P2 productivity:** calendar, email, lists; 10 intents.
- **P3 ordering, travel:** takeaway food and transport; 6 intents; held out.
- **|…|:** "the number of".

How P3 was kept secret

> P3 was held out completely: its description was committed at the start, but every split, including development, was locked by a data-access object that raises an error until released with the hash of the freeze commit. All development used P1 and P2. The protocol, thresholds, budgets, every system's selected design and all code were then frozen in one commit, and only after that were the test sets and P3 released.

From the paper

In short: P3's data was locked by code that raises an error unless the freeze commit's fingerprint is given. All development used only P1 and P2. Everything was frozen in one commit before test data and P3 were opened.

Words used here:

- **Data-access object:** the piece of code that guards the data.
- **Freeze commit:** the single saved version (in git) where all rules and choices were locked.

IV-B. Model and environment

B. Model and environment

All LM calls, both for planning and at runtime, use Qwen2.5-3B-Instruct [17] quantised to 4 bits (Q4_K_M), served locally by Ollama 0.40.2 on a laptop with an NVIDIA RTX 3050 GPU (4 GB). Decoding uses temperature 0, seed 0, JSON-constrained output and at most 512 output tokens. Fifty repeated uncached requests gave 50 identical outputs, so a single run per configuration is deterministic. For prompts averaging 1,028 input tokens a single call took 0.42 s at the median and 0.59 s at the 95th percentile. Model replies are cached by prompt hash for reproducibility; cached replies never contribute a latency figure.

From the paper

In short: Which AI and computer: Qwen2.5-3B-Instruct, compressed to 4 bits, run by Ollama on a laptop with an RTX 3050 (4 GB). Settings make it give the same answer every time (50 of 50 identical). One call takes 0.42 s typically.

Words used here:

- **Qwen2.5-3B-Instruct [17]:** the AI model (by Alibaba's Qwen team).
- **Q4_K_M:** the 4-bit compression format.
- **Ollama:** the free program that runs the model on the laptop.
- **Seed 0:** fixed randomness setting.
- **Deterministic:** same input, same output, every time.
- **Cached by prompt hash:** answers saved so experiments can be replayed; never used for timing.

IV-C. Baselines (the systems we compare against)

C. Baselines

B0 is TF-IDF features with logistic regression trained on K, with an OOS threshold tuned on V; it predicts no entities. **B1** is a generic prompt: the joint, fixed-example, model-label design on the bare specification, i.e., what a developer gets without any design effort. **B2** is a human designer: a team member studied the development results through a worksheet and chose one design (38 minutes on P1; under 6 minutes on P2, assisted by a chat assistant, which we declare); B2 was not run on P3. **B3** fine-tunes XLM-R base [2] with a joint intent and slot head, learning rate 3×10^{-5} and batch size 16, either on K for 30 epochs (B3-k) or on the full training split of the project's scenarios for 5 epochs (B3-full: 1,159, 3,180 and 828 utterances); its rejection threshold on maximum softmax probability is tuned on V. **B4-random** draws four designs uniformly at random and selects with the agent's rule, repeated with five seeds; **B4-best** evaluates all twelve designs and represents the best achievable choice on development data.

From the paper

In short: B0 = simple word-based classifier. B1 = generic prompt (no design effort). B2 = a person choosing the design. B3 = XLM-R trained on the same 10 examples (B3-k) or all training data (B3-full). B4 = 4 random designs (5 times); B4-best = best of all 12 on practice data.

Words used here:

- **Logistic regression:** a simple, classic classifier.
- **Bare specification:** the input without the AI-written descriptions.
- **Worksheet:** the form the person used to choose a design.
- **Epochs:** passes over the training data.
- **Learning rate 3×10^{-5}, batch size 16:** training settings (step size; 16 sentences at a time).
- **Maximum softmax probability:** the trained model's top confidence; low means maybe off-topic.

IV-D. Statistics and the success rule

D. Statistical protocol and success rule

Differences between two systems are tested with a paired bootstrap over test utterances [18]: 1,000 resamples with a fixed seed, both systems scored on the same resample, and the 2.5th and 97.5th percentiles of the difference as the 95% interval. A difference is called significant when the interval excludes zero. The success rule was fixed before testing: an agent succeeds against B1 if intent macro-F1 improves by at least 2.0 points, entity F1 drops by at most 2.0 points, schema validity is at least 0.98, and test-time 95th-percentile latency is at most twice B1's measured in the same session. Latency was measured on 50 development requests per system after five warm-up requests, with the systems interleaved request by request so that any slowdown of the machine affects all of them equally.

From the paper

In short: How we decide if a difference is real: resample the test set 1,000 times (paired bootstrap) and look at the middle 95% of differences. The success rule (fixed in advance): +2.0 points intent, at most −2.0 entity, valid replies ≥ 0.98, and no slower than twice B1. Timing alternates systems request by request so it is fair.

Words used here:

- **Paired bootstrap [18]:** draw test sets with replacement, score both systems on the same draw, repeat.
- **Percentile:** the value below which that share of results lies.
- **95% interval:** the range where the true difference probably is.
- **Significant:** the interval does not include zero.
- **Warm-up requests:** first requests not timed, so start-up slowness doesn't count.
- **Interleaved:** B1, A1, A2, B1, A1, A2… in turn.

Pages 6–8: results

Table III: the main test results

TABLE III
TEST RESULTS. INTENT MACRO-F1 / ENTITY SPAN F1 / OOS F1. BEST LM-BASED RESULT PER COLUMN IN BOLD. B4-RANDOM REPORTS THE MEAN INTENT MACRO-F1 OF FIVE SEEDS, WITH THE RANGE.

System	P1 home assistant	P2 productivity	P3 ordering and travel (held out)
B0 TF-IDF + logistic regression	0.699 / – / 0.426	0.669 / – / 0.403	0.769 / – / 0.404
B1 generic prompt	0.876 / 0.626 / 0.875	0.823 / 0.477 / 0.662	**0.895** / 0.529 / 0.771
B2 human designer	0.837 / 0.648 / 0.471	0.851 / 0.491 / 0.748	not run
B3-k XLM-R, same 10 per intent	0.809 / 0.650 / 0.568	0.800 / 0.461 / 0.536	0.770 / 0.515 / 0.505
B3-full XLM-R, full training split	0.883 / 0.802 / 0.604	0.904 / 0.775 / 0.560	0.913 / 0.721 / 0.581
B4-random, mean (range)	0.893 (0.881–0.903)	0.854 (0.848–0.872)	0.894 (0.882–0.909)
A1 agent, direct planner	**0.903** / 0.639 / **0.885**	0.852 / 0.504 / 0.758	0.889 / 0.506 / **0.845**
A2 agent, factor-wise planner	0.898 / **0.699** / 0.865	**0.872** / **0.532** / **0.767**	0.882 / **0.675** / 0.795

From the paper

In short: The most important table. Each cell is intent F1 / entity F1 / OOS F1 on locked test data. Bold = best AI-based result per column. A2 has the best entity score of the AI systems on all three projects; B1 has the best intent score on P3; B3-full (lots of training data) has the best entities overall but weak refusal.

Words used here:

- **–:** B0 does not predict entities.
- **Mean (range):** average of 5 random-search runs, with lowest and highest.
- **Held out:** P3, never used during development.

IV-E. Overall results: which designs were chosen

E. Overall results

Table III and Fig. 2 give the test results. The selected designs were: A2 schema-guided with similarity examples and model rejection on P1 and P2, and joint with similarity examples and model rejection on P3; A1 two-stage with similarity examples on P1 and schema-guided with fixed examples on P2 and P3, all with model rejection. In each project A2's choice coincides with B4-best, the best of all twelve designs on development data.

From the paper

In short: A2 chose schema-guided + similarity examples + AI rejection on P1 and P2, and joint + similarity + AI rejection on P3. Each time this equals the best of all 12 on practice data.

Words used here:

- **B4-best:** the best of all 12 designs on practice data (the ceiling).

Figure 2: intent scores as bars

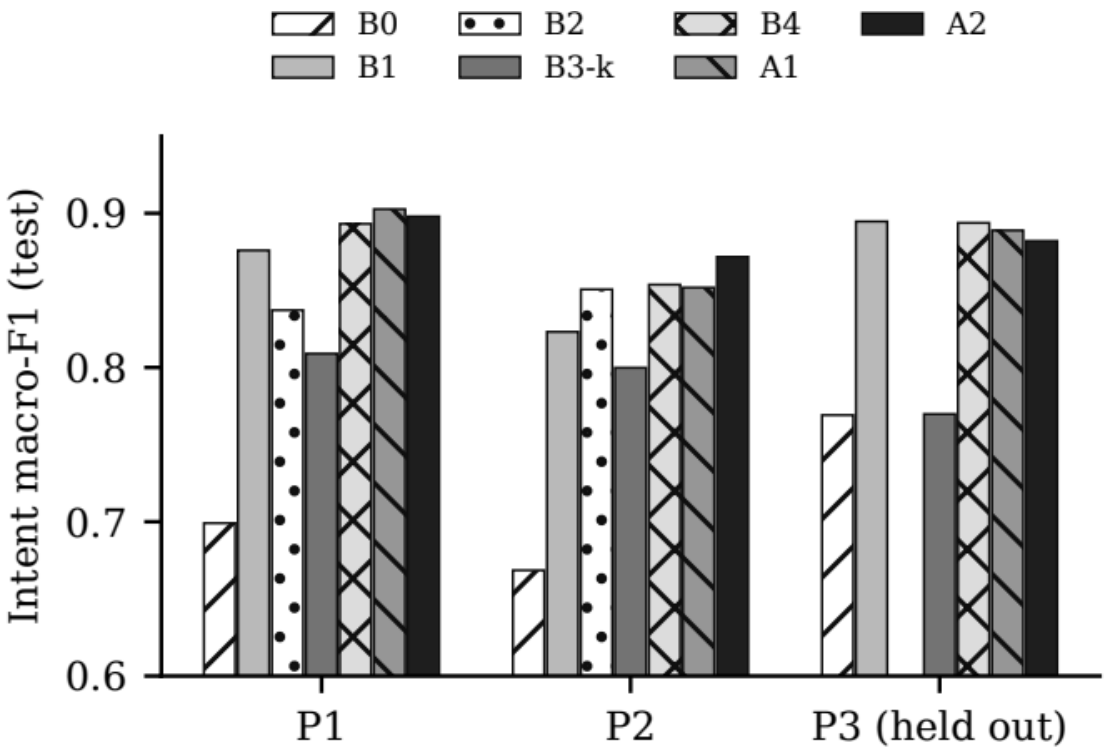


Fig. 2. Test intent macro-F1. B4 is the mean of five random-search seeds. B2 was not run on P3.

From the paper

In short: The intent numbers of Table III as a bar chart. Each pattern is one system. On P1 and P2 the agents (A1, A2) are among the tallest bars; on P3, B1 and the random-search average are slightly taller. The gap on P3 is B2 (not run there).

Words used here:

- **Intent macro-F1 (test):** the vertical axis; higher is better.

The success rule result

Success rule. Against B1, A2 changed intent / entity F1 by +2.2 / +7.3 points on P1, +4.9 / +5.5 on P2 and −1.3 / +14.6 on P3; A1 by +2.7 / +1.3, +2.9 / +2.7 and −0.6 / −2.3. Both agents therefore pass the rule on P1 and P2 and fail it on P3. Schema validity after repair was at least 0.997 in every agent run, and 95th-percentile latencies were within the limit (P1: A1 0.97 s, A2 0.89 s against 1.22 s; P2: A1 0.77 s, A2 1.08 s against 1.67 s). A2's median latency was 0.68 s and 0.64 s, against 0.45 s and 0.41 s for B1.

From the paper

In short: Compared with B1: A2 gained +2.2/+7.3 points (intent/entity) on P1, +4.9/+5.5 on P2, and −1.3/+14.6 on P3. So both agents pass on P1 and P2 and fail on P3. Valid replies ≥ 0.997, speed within the limit.

Words used here:

- **Pass / fail:** whether all four conditions of the rule were met.
- **Median latency:** typical time per request (A2 0.68 s vs B1 0.45 s on P1).

Significance: which differences are real

Significance. Fig. 3 shows the paired intervals for intent macro-F1. A2's gain over B1 is significant on P2 (+4.9, 95% CI [+1.1, +8.9]) but not on P1 (+2.2, [−0.6, +5.1]); on P3 the loss is not significant (−1.3, [−3.8, +1.4]). For entity F1, A2's gains over B1 are significant on all three projects: +7.3 [+0.8, +14.5] on P1, +5.5 [+0.3, +11.1] on P2 and +14.6 [+9.0, +21.3] on P3. None of A1's differences from B1 is significant. On test data 300 or 200 utterances give intervals about ±3 points wide, so a two-point improvement, the threshold of the success rule, can pass the rule without being significant, as on P1.

From the paper

In short: A2's intent gain over B1 is real (significant) on P2, not proven on P1, and the P3 loss is not significant. A2's entity gains are significant on all three. With only 200–300 test requests the intervals are about ±3 points wide, so a 2-point gain can pass the rule without being provable.

Words used here:

- **CI [+1.1, +8.9]:** the 95% interval of the difference in points.
- **n.s.:** not significant.

Figure 3: confidence intervals

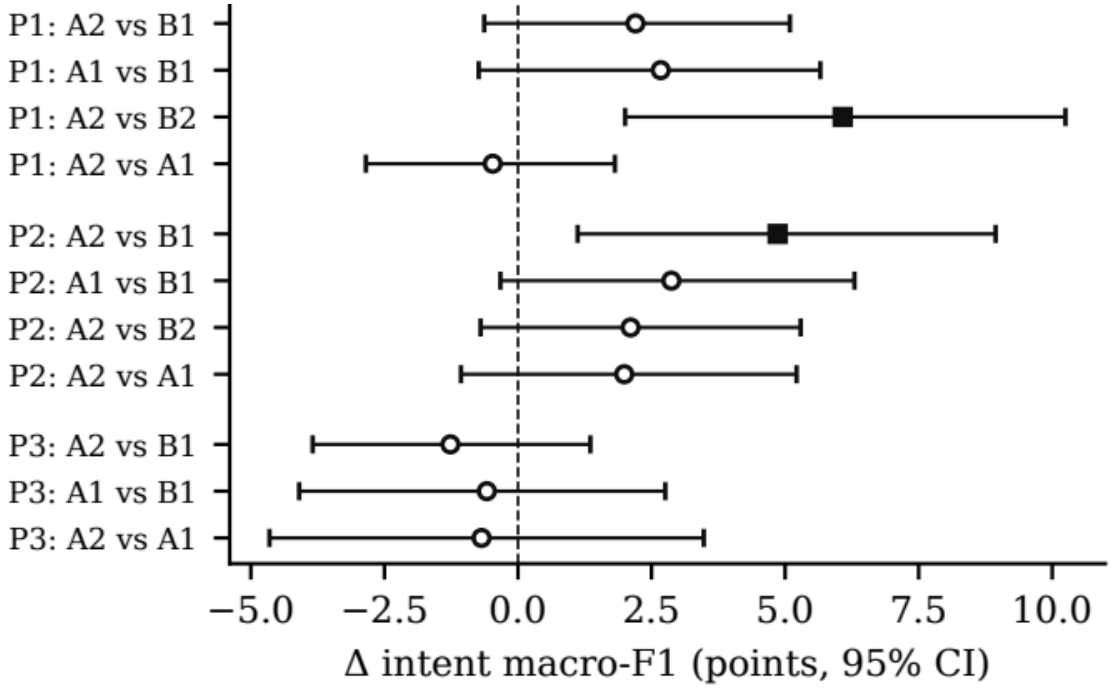


Fig. 3. Paired bootstrap 95% intervals of the difference in test intent macro-F1 (1,000 resamples). Filled squares: interval excludes zero.

From the paper

In short: Each line is one comparison. The dot is the measured difference; the line is the 95% range. If the line does not cross the dashed zero line (filled square), the difference is real. Only P1 "A2 vs B2" and P2 "A2 vs B1" clear zero for intent.

Words used here:

- **Δ (delta):** difference.
- **Filled square / open circle:** significant / not significant.

IV-F. Agent vs random search

F. Agent against random search and the human designer

Random search with the same budget and selection rule reached a mean intent F1 of 0.893, 0.854 and 0.894 on the three projects. A2 was above the mean on P1 (0.898) and P2 (0.872, the top of the random range), and at the bottom of the range on P3 (0.882). A random seed includes the development-optimal design among its four draws with probability $4/12$, and when it does, the shared selection rule picks it, so a seed can match A2 exactly. One of five seeds did so on P1 and P2, and two of five on P3. A2 reached that design on all three projects without depending on the draw.

From the paper

In short: Random search averaged 0.893, 0.854, 0.894. A2 was above that on P1, at the top on P2, at the bottom on P3. Random search finds the best practice design only by luck (1 of 5 runs on P1/P2, 2 of 5 on P3); A2 found it every time.

Words used here:

- **Probability 4/12:** chance that 4 random picks include one specific design.

Agent vs the human designer

Against the human designer, A2 was 6.1 points higher in intent F1 on P1 (95% CI [+2.0, +10.2]) and 2.1 points higher on P2 (n.s.). On P1 the human chose the threshold rejection policy, which reduced OOS F1 to 0.471 against A2's 0.865; the agent never kept a threshold design because on both development projects all six model-label designs scored above all six threshold designs.

From the paper

In short: A2 beat the person by 6.1 points on P1 (real) and 2.1 on P2 (not proven). On P1 the person chose threshold rejection, which made refusals much worse (0.471). On practice data every AI-rejection design beat every threshold design, so the agent never kept a threshold design.

IV-G. Agent vs trained XLM-R

G. Agent against fine-tuned encoders

Trained on the same ten utterances per intent, XLM-R (B3-k) was 8.9, 7.2 and 11.2 points below A2 in intent F1 and below it in entity F1 on all three projects. Its rejection by maximum softmax probability gave OOS F1 between 0.505 and 0.568, against 0.767 to 0.865 for A2. With the full training split, 10 to 32 times more labelled data, XLM-R (B3-full) exceeded A2 in entity F1 by 4.6 to 24.3 points and in intent F1 on P2 and P3, but its OOS F1 stayed below 0.61. With a few examples per intent the LM pipeline is the better choice, and its rejection was better in every data condition tested; with thousands of annotated utterances a fine-tuned encoder extracts spans better.

From the paper

In short: With the same 10 examples, XLM-R was 7–11 points worse on intents, worse on entities and much worse at refusing. With 10–32 times more data, XLM-R wins on entities but still refuses badly. So: few examples → use our pipeline; thousands of labelled sentences → a trained model is better at entities.

Words used here:

- **10 to 32 times more labelled data:** 1,159 / 3,180 / 828 training sentences vs 120 / 100 / 60.

IV-H. Why factor-wise planning worked

H. Why factor-wise planning worked

Table IV lists A2's four development candidates. On both P1 and P2 the planning model ranked fixed examples above similarity examples and the threshold policy above model rejection; on P1 its stated reason was that the threshold avoids model calls. Both preferences were wrong for these projects, and the first three candidates scored between 0.762 and 0.869. The construction of c_2 nevertheless measured similarity examples, which won, and the revision step, given the error report and the one-factor moves from c_2, switched its rejection policy to the model label. That single move raised development intent F1 by 10.5 points on P1 and 9.6 on P2 and reached the best of the twelve designs. A1, which names designs directly, began with B1's design on both projects, never combined schema guidance with similarity examples, and spent its unrestricted revision on a threshold design that scored 0.862 on P1 and 0.790 on P2; it stopped at 0.958 and 0.841. The gain comes from a structure that forces each factor to be tested and restricts revision to one-factor moves, not from better first guesses.

From the paper

In short: The key story. The AI's favourite options (fixed examples, threshold rejection) were both wrong, and the first three tries scored 0.76–0.87. But c2 always tests the other example option (similarity won), and the revision switched rejection to the AI label: +10.5 and +9.6 points, the best of all 12. A1 never found it. The structure, not the AI's guesses, made the difference.

Words used here:

- **Unrestricted revision:** A1 could pick any untried design, and picked a poor one.

Table IV: A2's four tries on practice data

TABLE IV
A2'S DEVELOPMENT TRACE: INTENT MACRO-F1 / ENTITY F1 OF EACH CANDIDATE (100 DEVELOPMENT UTTERANCES)

Step	Design	P1	P2
c_1	schema, fixed, threshold	0.845 / 0.505	0.762 / 0.391
c_2	schema, similarity, threshold	0.869 / 0.617	0.802 / 0.437
c_3	two-stage, fixed, threshold	0.839 / 0.544	0.772 / 0.413
c_4	schema, similarity, model label	**0.974 / 0.707**	**0.898 / 0.483**
Range over all 12 designs		0.839–0.974	0.738–0.898

From the paper

In short: The scores of c1, c2, c3 and the revised c4 on P1 and P2. c4 (schema, similarity, AI label) is the best and equals the top of the range of all 12 designs.

Words used here:

- **Range over all 12 designs:** worst to best when every design is tested.

Honesty note

> A2 was built after A1's development behaviour had been analysed, so its development scores on P1 and P2 are not independent evidence. Its test scores and all of P3 are: the design, the keep-or-drop rule and the code were frozen before either was released.

From the paper

In short: A2 was designed after we studied A1's practice results, so A2's practice scores are not independent proof. Its test scores and all of P3 are, because everything was frozen before those were opened.

Words used here:

- **Independent evidence:** results not influenced by the people who built the system.
- **Keep-or-drop rule:** a rule fixed in advance for whether A2 would be tested at all.

IV-I. Held-out project P3

I. Held-out project

On P3 A2 again picked the best development design, joint extraction with similarity examples and model rejection, but on test it scored 1.3 points below B1 and at the bottom of the random-search range. The development set of P3 has 80 in-scope utterances over 6 intents, about 13 per intent, and the three designs selected by the random seeds lay within 2.7 points of each other on test (0.882 to 0.909), so the development ranking did not carry over to test. The entity gain carried over strongly: A2's entity F1 was 14.6 points above B1 and 16.9 points above A1 (CI [+10.7, +23.5]), because similarity-retrieved demonstrations show the model slot values like the ones in the request.

From the paper

In short: On P3 A2 again picked the best practice design, but on test it was 1.3 points below B1. P3's practice set had only about 13 requests per intent and the designs were very close (0.882–0.909), so the practice ranking did not carry over. Entities improved strongly (+14.6 over B1).

IV-J. Ablations

J. Ablations

Table V removes one component of A1 at a time; each variant made its own choice on development data before testing. Removing selection (running the first candidate, which is B1's design) cost 2.7 and 2.9 points on P1 and P2, the largest effect, and gained 0.6 on P3. Removing the enrichment of analysis lowered P2 intent F1 by 1.1 and entity F1 by 7.4 points (0.504 to 0.430), consistent with P2's five entity types that never appear in the pool, for which the descriptions are the only information the model receives; this variant also chose a different design (two-stage with fixed examples). Removing the repair step changed intent F1 by at most 0.4 points, because the model rarely broke the schema.

From the paper

In short: Remove one part of A1 at a time to see what matters. Removing selection hurts most (−2.7, −2.9). Removing the descriptions hurts P2 entities a lot (−7.4) because some entity types have no examples. Removing repair barely matters because the AI rarely broke the format.

Words used here:

- **Ablation:** removing a component to measure its value.
- **Without selection:** just use the first candidate (which is B1's design).

Table V: ablation scores

TABLE V
ABLATIONS OF A1 ON TEST, INTENT MACRO-F1

Variant	P1	P2	P3
A1 (full)	0.903	0.852	0.889
without selection	0.876	0.823	0.895
without analysis enrichment	0.903	0.841	0.898
without planner reasoning	0.903	0.848	0.882
without schema repair	0.907	0.852	0.889

From the paper

In short: Intent F1 on test for A1 and each variant with one part removed, on all three projects.

Words used here:

- **Planner reasoning:** the AI's choice of designs (replaced by a fixed order in that variant).

IV-K. Cost and reproducibility

K. Cost and reproducibility

Designing one project with A2 cost about 397 logical model calls: two for enrichment, two for planning and about 393 for running four candidates on 100 development utterances (two-stage designs need two calls per request, and threshold rejections need none), about a third of exhaustive evaluation and about three minutes of model time per project, against 38 minutes for the human designer on P1. Every reported metric was recomputed from the saved predictions of its run: 99 of 99 runs reproduced exactly, and an independent implementation of intent macro-F1, accuracy and OOS precision, recall and F1 agreed to nine decimal places. Retraining B0 from scratch reproduced all its test metrics exactly.

From the paper

In short: Designing one project costs A2 about 397 AI calls (about 3 minutes), against 38 minutes for the person. Every number was recomputed from saved answers (99 of 99 runs matched), checked by an independent library to 9 decimal places, and B0 retrained from scratch gave identical results.

Words used here:

- **Logical model calls:** calls the method needs (counting cached ones too).
- **Exhaustive evaluation:** testing all 12 designs.
- **Reproduced exactly:** same numbers when recomputed.

Page 8: conclusion, acknowledgment, references

V. Conclusion

V. Conclusion

We built an agent that turns a project specification into a deployable, hash-verified intent and slot pipeline by evaluating four of twelve designs on 100 development utterances, using one 3-billion-parameter model on a laptop. Under a protocol frozen before the test data and the held-out project were seen, the factor-wise agent chose the development-optimal design on all three projects and met the preregistered success rule on both development projects, with intent gains of 2.2 and 4.9 points over a generic prompt. Its entity gains were significant on all three projects, including 14.6 points on the held-out project, where intent F1 fell by a non-significant 1.3 points. It scored above XLM-R trained on the same examples on every project. The gain came from planning that tests every factor and revises one at a time, not from the model's first preferences.

From the paper

In short: A short restatement of what was built and found: best practice design on all three projects, success on P1 and P2, significant entity gains everywhere, the honest P3 result, better than XLM-R on the same data, and why it worked. No limitations or future work here (our teacher's rule).

Acknowledgment

Acknowledgment

The authors thank Vijayaprabakaran K for guidance throughout this work. MASSIVE is used under the CC BY 4.0 licence. An AI assistant (Claude, Anthropic) helped with code, analysis and drafting; the authors ran all experiments, checked every number against the saved outputs and take full responsibility for the content.

From the paper

In short: Thanks the guide, states the dataset licence, and declares the use of an AI assistant.

References

REFERENCES

[1] Q. Chen, Z. Zhuo, and W. Wang, "BERT for joint intent classification and slot filling," *arXiv preprint arXiv:1902.10909*, 2019.

[2] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised cross-lingual representation learning at scale," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2020, pp. 8440–8451.

From the paper

In short: The 18 papers cited, in IEEE style, numbered in the order they first appear in the text. Each entry gives authors, title, where it was published, year and pages.

Words used here:

- **arXiv preprint:** a paper posted online before or without formal publication.
- **Proceedings of …:** the book of papers from a conference (ACL, EMNLP, ICLR, NeurIPS are top AI/NLP conferences).
- **vol., pp.:** volume and page numbers.

[3] J. FitzGerald, C. Hench, C. Peris, S. Mackie, K. Rottmann, A. Sanchez, A. Nash, L. Urbach, V. Kakarala, R. Singh, S. Ranganath, L. Crist, M. Britan, W. Leeuwis, G. Tur, and P. Natarajan, "MASSIVE: A 1M-example multilingual natural language understanding dataset with 51 typologically-diverse languages," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Toronto, Canada: Association for Computational Linguistics, 2023, pp. 4277–4302.

[4] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, "Large language models are human-level prompt engineers," in *International Conference on Learning Representations (ICLR)*, 2023.

[5] R. Pryzant, D. Iter, J. Li, Y. T. Lee, C. Zhu, and M. Zeng, "Automatic prompt optimization with "gradient descent" and beam search," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Singapore: Association for Computational Linguistics, 2023, pp. 7957–7968.

[6] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," in *International Conference on Learning Representations (ICLR)*, 2024.

[7] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "DSPy: Compiling declarative language model calls into state-of-the-art pipelines," in *International Conference on Learning Representations (ICLR)*, 2024.

[8] V. Viswanathan, C. Zhao, A. Bertsch, T. Wu, and G. Neubig, "Prompt2Model: Generating deployable models from natural language instructions," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Singapore: Association for Computational Linguistics, 2023, pp. 413–421.

[9] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.

[10] E. Bastianelli, A. Vanzo, P. Swietojanski, and V. Rieser, "SLURP: A spoken language understanding resource package," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2020, pp. 7252–7262.

[11] S. Larson, A. Mahendran, J. J. Peper, C. Clarke, A. Lee, P. Hill, J. K. Kummerfeld, K. Leach, M. A. Laurenzano, L. Tang, and J. Mars, "An evaluation dataset for intent classification and out-of-scope prediction," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, 2019, pp. 1311–1316.

[12] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

Processing Systems, vol. 33, 2020.

[13] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin, and W. Chen, "What makes good in-context examples for GPT-3?" in *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures*. Dublin, Ireland: Association for Computational Linguistics, 2022, pp. 100–114.

[14] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[15] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[16] X. He, K. Zhao, and X. Chu, "AutoML: A survey of the state-of-the-art," *Knowledge-Based Systems*, vol. 212, p. 106622, 2021.

[17] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[18] P. Koehn, "Statistical significance tests for machine translation evaluation," in *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*. Barcelona, Spain: Association for Computational Linguistics, 2004, pp. 388–395.

From the paper: references, second column